

EnergyComm — Memoria Técnica

Asignatura: Aplicaciones Web (ALS)

Autor: Rodríguez Núñez, Roberto

Profesor: Baltasar García Pérez-Schofield

Curso: 2025/2026 · Universidade de Vigo

1. Visión general

EnergyComm es una plataforma SaaS multi-tenant para la gestión de comunidades energéticas. Una comunidad energética es un grupo de viviendas que comparten una batería de almacenamiento y, opcionalmente, instalaciones fotovoltaicas individuales.

El proyecto es la capa de gestión administrativa del TFG del autor, que implementa un agente de Reinforcement Learning que optimiza el flujo energético entre 15 casas. El SaaS gestiona los datos de entrada del agente (viviendas, coeficientes de reparto, batería) y presenta los resultados del agente (CierresMensuales) a través de una interfaz web multi-rol.

2. Stack tecnológico

Tecnología	Versión	Rol
Python	3.11+	Lenguaje principal
Flask	3.0.3	Framework web
Jinja2	(incluido en Flask)	Plantillas HTML
Sirope	0.3.1	ORM sobre Redis (persistencia)
Redis	7	Base de datos
Flask-Login	0.6.3	Gestión de sesiones
Flask-WTF	1.2.1	Protección CSRF y formularios

WTForms	3.1.2	Validación de formularios
Pico CSS	2 (CDN)	Framework CSS, tema oscuro
Chart.js	4 (CDN)	Gráficas interactivas

3. Arquitectura

3.1. Patrón Application Factory

La aplicación usa el patrón *Application Factory* (`app/__init__.py`). Ventajas: configuración inyectable, múltiples instancias posibles (tests, producción).

3.2. Blueprints modulares

Un blueprint por entidad o grupo funcional, siguiendo el criterio del enunciado de un módulo por entidad:

Blueprint	Prefijo	Entidad
`auth`	`/`	Autenticación
`main`	`/`	Dashboard según rol
`comunidades`	`/comunidades`	Comunidad
`viviendas`	`/viviendas`	Vivienda
`usuarios`	`/usuarios`	Usuario
`accesos`	`/accesos`	AccesoVivienda
`baterias`	`/baterias`	Bateria
`cierres`	`/cierres`	CierreMensual
`notificaciones`	`/notificaciones`	Notificacion

3.3. OIDs URL-safe

Sirope 0.3.1 genera OIDs con formato `namespace@numero` (ej: `app.models.comunidad.Comunidad@3`). Los puntos y la arroba no son seguros en parámetros de ruta Flask. Solución adoptada:

```
def oid_to_safe(oid) -> str:    return str(oid).replace('.', '-dot-').replace('@', '-at-')def
```

Esta función se registra como global Jinja2 (`app.jinja_env.globals['oid_to_safe'] = oid_to_safe`) para uso directo en templates.

4. Modelo de datos

4.1. Entidades

- Comunidad** — Agrupación de viviendas con infraestructura compartida.
- Vivienda** — Unidad habitacional dentro de una comunidad, con o sin paneles solares.
- Usuario** — Persona con cuenta en la plataforma (superadmin o normal).
- AccesoVivienda** — Relación N:M entre Usuario y Vivienda con rol.
- Bateria** — Batería de almacenamiento de una comunidad (máximo una por comunidad).
- CierreMensual** — Datos energéticos y económicos mensuales de una vivienda.
- Notificacion** — Mensajes del sistema para usuarios.

4.2. Relaciones



4.3. Almacenamiento en Sirope

Sirope serializa objetos Python como JSON en Redis. No hay esquema fijo: las relaciones se mantienen como strings OID en los atributos de los objetos (ej. `vivienda.comunidad_oid = str(comunidad_oid)`). La integridad referencial es responsabilidad del código de la aplicación.

5. Roles y permisos

Operación	Superadmin	Admin comunidad	Usuario normal
CRUD comunidades	✓	✗	✗
Ver comunidades	✓	Solo las suyas	Solo las suyas

CRUD viviendas	✓	Solo su comunidad	✗
CRUD accesos	✓	Solo su comunidad	✗
CRUD batería	✓	Solo su comunidad	✗
CRUD cierres	✓	Solo su comunidad	✗
Ver cierres	✓	✓	Solo sus viviendas
Notificaciones	✓ (todas)	Solo las suyas	Solo las suyas

Un usuario es admin de comunidad si tiene un `AccesoVivienda` con `es_admin_comunidad=True` en alguna vivienda de esa comunidad.

5.1. Implementación

- `app/decorators.py`: `@superadmin_required`, `@admin_comunidad_required()`, `@acceso_vivienda_required()`
- `app/helpers.py`: `es_admin_de_comunidad(srp, usuario_oid, comunidad_oid)`, `usuario_tiene_acceso(srp, usuario_oid, vivienda_oid)`

6. Integridad referencial

Decisión explícita del diseñador (criterio del enunciado):

Borrar	Guardia	Efecto
Comunidad	Solo superadmin. Confirmación doble.	Cascada: viviendas → (accesos + cierres) → batería → notificaciones relacionadas
Vivienda	Admin o superadmin	Cascada: accesos de esa vivienda + cierres de esa vivienda
Usuario	Falla si último titular o último admin	Borrar sus accesos primero
AccesoVivienda	Falla si último titular	Borrado simple
Batería	Admin o superadmin	Borrado simple (comunidad queda sin batería)
CierreMensual	Admin o superadmin	Borrado simple
Notificación	Propietario o superadmin	Borrado simple

Las funciones de cascada están en `app/helpers.py`: `cascade_delete_comunidad`, `cascade_delete_vivienda`.

La guardia de "último titular" se ejecuta en la ruta de borrar AccesoVivienda antes de llamar a `srp.delete()`.

7. AJAX Progressive Enhancement

Todos los formularios de borrado (y marcar-como-leída) funcionan en dos modos:

Sin JavaScript: POST estándar → el servidor responde con `flash()` + `redirect()`.

Con JavaScript: `ajax.js` intercepta `form[data-ajax-action]`, envía `fetch()` con `X-Requested-With: XMLHttpRequest`, y el servidor devuelve JSON. El DOM se actualiza sin recargar.

Detección en servidor:

```
def is_xhr(req=None) -> bool:    return req.headers.get('X-Requested-With') == 'XMLHttpRequest'
```

CSRF en AJAX: el token se lee del `__csrf_token__` y se incluye en el FormData.

Configuración Flask-WTF: `WTF_CSRF_HEADERS = ['X-CSRFToken']`.

Acciones con AJAX: borrado de todas las entidades, marcar notificación como leída, cambiar estado de batería.

8. Diseño visual

- **Pico CSS 2** en modo oscuro permanente ()
- **Paleta:** verde eléctrico `#00d4aa` (primario), amarillo `#ffc857` (acento), rojo `#ff4d6d` (alertas), fondo `#0b1117`
- **Navbar** sticky con glassmorphism (`backdrop-filter: blur(12px)`)
- **Flash messages** con borde izquierdo del color del tipo de mensaje
- **Chart.js 4** para gráficas: línea de ahorro mensual en detalle de vivienda, doughnut de distribución de energía en detalle de cierre

9. Conexión con el TFG

El módulo `CierreMensual` es el punto de integración entre el SaaS y el agente RL del TFG:

- **Entrada del agente:** datos de Vivienda (potencia contratada, coeficiente de reparto, paneles) y Bateria (capacidad, ciclos).
- **Salida del agente:** los campos `consumo_total_kwh`, `autoconsumo_directo_kwh`, `energia_de_bateria_kwh`, `vertido_a_red_kwh`, `ahorro_eur`, etc.

En el SaaS académico estos datos se introducen manualmente. En el sistema de TFG, el agente generaría estos datos automáticamente y los insertaría vía la API interna.

10. Criterios del enunciado — Tabla de cumplimiento

Criterio	Implementación
Flask + Jinja2	✔ Flask 3.0.3, templates Jinja2
Sirope + Redis	✔ Sirope 0.3.1, Redis 7 vía Docker
Flask-Login	✔ Autenticación completa con sesión segura
Flask-WTF	✔ CSRF en todos los formularios
Diseño modular	✔ 9 blueprints, 7 modelos, un módulo por entidad
Pico CSS	✔ Pico CSS 2, dark theme permanente
Sin recarga (AJAX)	✔ `ajax.js` con progressive enhancement
Robustez	✔ 404/403/500 amigables, errores flasheados
Integridad referencial	✔ Documentada y implementada explícitamente
Navegable sin volver atrás	✔ Breadcrumbs en todas las vistas
Primer usuario es superadmin	✔ `auth.registro` comprueba `num_objs(Usuario) == 0`